

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

on

Artificial Intelligence (23CS5PCAIN)

Submitted by

AFEEFAH (1BM23CS015)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Aug-2025 to Dec-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Artificial Intelligence (23CS5PCAIN)” carried out by **Afeefah (1BM23CS015)**, who is a bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Artificial Intelligence (23CS5PCAIN) work prescribed for the said degree.

SWATHI SRIDHARAN Associate Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
--	---

Index

Sl. No.	Date	Experiment Title	Page No.
1	20-8-2025	Implement Tic –Tac –Toe Game Implement vacuum cleaner agent	4
2	3-9-2025	Implement 8 puzzle problems using Depth First Search (DFS) Implement Iterative deepening search algorithm	12
3	10-9-2025	Implement A* search algorithm	20
4	8-10-2025	Implement Hill Climbing search algorithm to solve N-Queens problem	24
5	8-10-2025	Simulated Annealing to Solve 8-Queens problem	27
6	15-10-2025	Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.	30
7	29-10-2025	Implement unification in first order logic	33
8	29-10-2025	Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.	36
9	12-11-2025	Create a knowledge base consisting of first order logic statements and prove the given query using Resolution.	38
10	12-11-2025	Implement Alpha-Beta Pruning.	45

Github Link:

<https://github.com/Afeefahafeefah/5th-SEM-LABS/tree/main/AI%20LAB>



Program 1

Implement Tic - Tac - Toe Game

Implement vacuum cleaner agent

Algorithm:

LAB- 9

TIC TAC TOE PROBLEM:

player - X, System - O

- 1) create a 3x3 board with positions 1-9.
- 2) make a function displayboard().
- 3) if any row or column has symbol X or O $\Rightarrow$ return ^{winning}
- 4) Function checkwin() - checks if a row, column or a diagonal has same 3 consecutive symbols
- 5) The 3x3 grid is represented as follows:
0 1 2
3 4 5
6 7 8

The player is asked to input number and symbol.

- 6) prints "invalid move, try again" if position is invalid
- 7) After a series of moves $\Rightarrow$ returns() to see which player won
- 8) is full() - return draw case
- 9) player() - ask position and place X at position
- 10) ~~computer move~~
best-move() - choose the best available move and place O.

Output for Tic Tac Toe

```

enter pos(0-8): 1 4 3
enter pos(0-8): 2 8
enter pos(0-8): 6
Its a draw.

```

1	4	3
2	8	
6		

Its a draw.

pseudo code:
 board = [" " for i in range(9)]
 def print_board(board):
 for i in range(3):
 print(board[i*3:(i+1)*3])
 def available_moves(board):
 return [i for i in range(9) if board[i] == " "]
 def winner(board):
 win_cond = [[0,1,2], [3,4,5], [6,7,8],
 [0,3,6], [1,4,7], [2,5,8],
 [0,4,8], [2,4,6]]
 for combo in win_cond:
 if board[combo[0]] == board[combo[1]] == board[combo[2]] != " ":
 return board[combo[0]]
 if no empty spaces:
 return "Tie"
 return None
 def minimax(board, depth, is_maximizing):
 result = winner(board)
 if result == "X": return -1
 if result == "O": return 1
 if result == "Tie": return 0
 if is_maximizing:
 best_score = -infinity
 for move in available_moves(board):
 board[move] = "X"
 score = minimax(board, depth+1, False)
 board[move] = " "
 best_score = max(score, best_score)
 return best_score
 else:

else:
 best_score = +infinity
 for move in available_moves(board):
 board[move] = "O"
 score = minimax(board, depth+1, True)
 board[move] = " "
 best_score = min(score, best_score)
 return best_score
 def best_move(board):
 best_score = -infinity
 move = None
 for each available move:
 board[move] = "O"
 score = minimax(board, 0, False)
 board[move] = " "
 if score > best_score:
 best_score = score
 move = current move
 return move
 def play():
 print("Tic Tac Toe (You are X, AI is O)")
 print_board(board)
 while True:
 ask user for input (0-8)
 if invalid move: continue
 place "X" on board
 print_board(board)
 if winner exists: announce winner; break
 or move = best_move(board)
 place "O" on board
 print_board(board)
 if winner exists: announce winner; break
 play()

O/P

TIC-TAC-TOE :

Code:

```
def print_board(board):
    print("\n 0  1  2")
    for i in range(3):
        print(f"{i} {board[i][0]} | {board[i][1]} | {board[i][2]}")
        if i < 2:
            print(" -----")
    print()

def check_winner(board):    # Check
    rows    for row in board:    if row[0]
    == row[1] == row[2] != '-':
        return row[0]

    # Check columns    for col in range(3):    if
    board[0][col] == board[1][col] == board[2][col] != '-':
        return board[0][col]

    # Check diagonals    if board[0][0] ==
    board[1][1] == board[2][2] != '-':
        return board[0][0]    if board[0][2] ==
    board[1][1] == board[2][0] != '-':
        return board[0][2]

    return None

def is_board_full(board):
    for row in board:
        if
        '-' in row:
            return
    False
    return True

def is_valid_move(board, row, col):
    return 0 <= row < 3 and 0 <= col < 3 and board[row][col] == '-'

def get_player_move(player):
    while True:
        try:
            move = input(f"Player {player}, enter your move (row col): ")
            row, col = map(int,
            move.split())
            return row, col
        except (ValueError, IndexError):
            print("Invalid input! Please enter row and column as two numbers (0-2).")
```



```

def play_tic_tac_toe():
    # Initialize empty board
    board = [['-' for _ in range(3)] for _ in range(3)]
    current_player = 'X'

    print("Welcome to Tic-Tac-Toe!")
    print("Enter moves as 'row col' ")
    print("Positions are numbered 0, 1, 2")

    # Main game loop
    while True:
        print_board(board)

        # Get player move
        row, col = get_player_move(current_player)

        # Check if move is valid
        if is_valid_move(board, row, col):
            # Make the move
            board[row][col] = current_player

            # Check for winner
            winner = check_winner(board)
            if winner:
                print_board(board)
                print(f"Player {winner} wins!")
                break

            # Check for tie
            if is_board_full(board):
                print_board(board)
                print("It's a tie!")
                break

            # Switch players
            current_player = 'O' if current_player == 'X' else 'X'
            print("Invalid move! That position is taken or out of bounds.")

def main():
    while True:
        play_tic_tac_toe()

        # Ask to play again
        play_again = input("\nDo you want to play again? (y/n): ").lower()
        if play_again != 'y':
            print("Thanks for playing!")

```

break

```
# Run the game if  
__name__ == "__main__":  
main()
```

Output:

```

Welcome to Tic-Tac-Toe!
Enter moves as 'row col' (e.g., '1 2')
Positions are numbered 0, 1, 2

  0  1  2
0 - | - | -
-----
  1  2  3
1 - | - | -
-----
  2  3  4
2 - | - | -

Player X, enter your move (row col): 0 1

  0  1  2
0 - | X | -
-----
  1  2  3
1 - | - | -
-----
  2  3  4
2 - | - | -

Player O, enter your move (row col): 1 1

  0  1  2
0 - | X | -
-----
  1  2  3
1 - | O | -
-----
  2  3  4
2 - | - | -

Player X, enter your move (row col): 0 0

  0  1  2
0 X | X | -
-----
  1  2  3
1 - | O | -
-----
  2  3  4
2 - | - | -

Player O, enter your move (row col): 1 1
Invalid move! That position is taken or out of bounds.

  0  1  2
0 X | X | -
-----
  1  2  3
1 - | O | -
-----
  2  3  4
2 - | - | -

Player O, enter your move (row col): 2 2

  0  1  2
0 X | X | -
-----
  1  2  3
1 - | O | -
-----
  2  3  4
2 - | - | O

Player X, enter your move (row col): 0 2

  0  1  2
0 X | X | X
-----
  1  2  3
1 - | O | -
-----
  2  3  4
2 - | - | O

🎉 Player X wins!

Do you want to play again? (y/n): n
Thanks for playing!

```

Vacuum cleaner

```
import random
```

```

# Function to clean a room
def clean_room(rooms, position):
    if rooms[position] == 1:
        print(f"Room {position+1} is Dirty. Cleaning...")
        rooms[position] = 0
        print(f"Room {position+1} is now Clean.")
    else:
        print(f"Room {position+1} is already Clean.")

# Function to move to the next room
def move(position, total_rooms):
    position = (position + 1) % total_rooms
    print(f"Moving to Room {position+1}")
    return position

# Function to run the vacuum cleaner
def run(rooms, steps):
    position = 0 # start at first room
    for _ in range(steps):
        clean_room(rooms, position)
    position = move(position, len(rooms))
    print("Final Room Status:", rooms)

# Initialize 4 rooms randomly (0 = clean, 1 = dirty)
rooms = [random.choice([0, 1]) for _ in range(4)]
print("Initial Room Status:", rooms)

# Run for 8 steps
run(rooms, 8)

```

Output:

```

Initial Room Status: [0, 1, 0, 1]
Room 1 is already Clean.
Moving to Room 2
Room 2 is Dirty. Cleaning...
Room 2 is now Clean.
Moving to Room 3
Room 3 is already Clean.
Moving to Room 4
Room 4 is Dirty. Cleaning...
Room 4 is now Clean.
Moving to Room 1
Room 1 is already Clean.
Moving to Room 2
Room 2 is already Clean.
Moving to Room 3
Room 3 is already Clean.
Moving to Room 4
Room 4 is already Clean.
Moving to Room 1
Final Room Status: [0, 0, 0, 0]

```

Program 2

Implement 8 puzzle problems using Depth First Search (DFS)

Implement Iterative deepening search algorithm

Algorithm:

Lab-2

8. PUZZLE USING MISPLACED TILES:

Algorithm solvesPuzzle (start-state, goal-state)

Input: start-state, goal-state

Output: sequence of moves from start-state to goal-state

1. Initialize:

- open-list $\leftarrow$ priority queue
- closed-list $\leftarrow$ empty set
- $g(\text{start-state}) \leftarrow 0$
- $h(\text{start-state}) \leftarrow$ misplaced Tiles (start-state, goal-state)

$f(\text{start-state}) \leftarrow g+h$

Add start-state to open-list

2. while open-list is not empty:

- current-node $\leftarrow$ node with lowest f in open-list
- Remove current-node from open-list
- Add current-node.state to closed-list

If current-node.state = goal-state

Return path from start-state to goal-state

For each valid move from current-node

new-state $\leftarrow$ result of move

If new-state $\in$ closed-list:

Continue

$g(\text{new-state}) \leftarrow g(\text{current-node}) + 1$

$h(\text{new-state}) \leftarrow$ misplaced Tiles (new-state, goal-state)

$f(\text{new-state}) \leftarrow g+h$

Add new-state to open-list

3. If open-list is empty:

Print "No solution exists"

End Algorithm

Function Misplaced Tiles (state, goal-state)

count $\leftarrow 0$

For i from 0 to 8:

If state[i] $\neq$ goal-state[i] And state[i] $\neq 0$:

count ++

Return count

output:-

$h(2,3) = 0$

$h(4,5,6) = 2$

$h(7,8) = 1$

$f = g+h = 2$

Diagram illustrating the search space for the 8-puzzle problem:

Root node: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{bmatrix}$

Children of Root:

- Node 1: $\begin{bmatrix} 1 & 2 & 3 \\ 2 & 5 & 6 \\ 4 & 7 & 8 \end{bmatrix}$
- Node 2: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 0 & 7 & 8 \end{bmatrix}$

Children of Node 1:

- Node 3: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{bmatrix}$
- Node 4: $\begin{bmatrix} 1 & 2 & 3 \\ 2 & 5 & 6 \\ 4 & 7 & 8 \end{bmatrix}$

Children of Node 2:

- Node 5: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 0 & 7 & 8 \end{bmatrix}$
- Node 6: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{bmatrix}$

8- Puzzle using MANHATTAN DISTANCE

Algorithm Solve 8-Puzzle Manhattan (start-state, goal-state)

Input: start-state, goal-state
Output: sequence of moves from start-state to goal-state

1. Initiation:

open-list $\leftarrow$ priority queue

closed-list $\leftarrow$ empty set

$g(\text{start-state}) \leftarrow 0$

$h(\text{start-state}) \leftarrow \text{Manhattan Distance}(\text{start-state}, \text{goal-state})$

$f(\text{start-state}) \leftarrow g + h$

Add start-state to open-list

2. while open-list is not empty:

current-node $\leftarrow$ node in open-list with lowest f

Remove current-node from open-list

Add current-node state to closed-list

3. if current-node state = goal-state:

Return path from start-state to goal-state

For each valid move (up, down, left, right):

new-state $\leftarrow$ result of move

If new-state $\in$ closed-list:

continue

$g(\text{new-state}) \leftarrow g(\text{current-node}) + 1$

$h(\text{new-state}) \leftarrow \text{Manhattan Distance}(\text{new-state}, \text{goal-state})$

$f(\text{new-state}) \leftarrow g(\text{new-state}) + h(\text{new-state})$

Add new-state to open-list

3. If open-list is empty:
print "No solution exists"

End Algorithm

function ManhattanDistance(state, goal-state)

distance $\leftarrow 0$

for each tile in state:

1. $dist \leftarrow 0$

current-position $\leftarrow$ index of tile in state

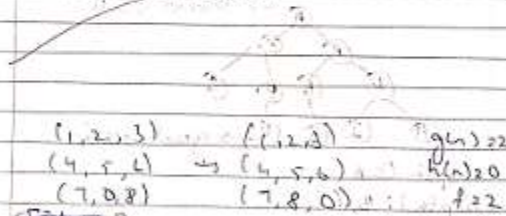
goal-position $\leftarrow$ index of tile in goal-state

row-diff $\leftarrow | \text{current-row} - \text{goal-row} |$

col-diff $\leftarrow | \text{current-col} - \text{goal-col} |$

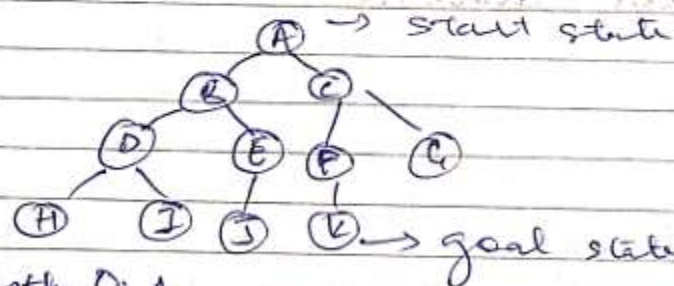
distance $\leftarrow \text{distance} + \text{row-diff} + \text{col-diff}$

Return distance



IDDFS

```
def iddfs(root, goal, graph, max_depth):
    def dfs(node, depth, path):
        if node == goal:
            return path
        if depth == 0:
            return None
        for child in graph.get(node, []):
            result = dfs(child, depth-1, path+[child])
            if result:
                return result
        return None
    for i in range(max_depth+1):
        result = dfs(root, i, [root])
        if result:
            return result
    return None
```



Depth 0: A

Depth 1: B, C

Depth 2: D, E, F, G

Depth 3: K found

path = A → C → F → K

Code:

Misplaced Tiles – 8 puzzle import
heapq

```

# ----- Heuristic: Misplaced Tiles ----- def
misplaced_tiles(state, goal):
    """Count number of misplaced tiles (ignores blank 0)."""
    return sum(1 for i in range(len(state)) if state[i] != 0 and state[i] != goal[i])

# ----- Puzzle Neighbors ----- def
get_neighbors(state):
    neighbors = []
    idx = state.index(0) #
    position of blank    x, y = divmod(idx, 3)

    moves = [(-1,0),(1,0),(0,-1),(0,1)] # up, down, left, right
    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if
        0 <= nx < 3 and 0 <= ny < 3:
            new_idx = nx * 3 + ny
            new_state = list(state)
            new_state[idx], new_state[new_idx] = new_state[new_idx], new_state[idx]
            neighbors.append(tuple(new_state))
    return neighbors

# ----- A* Search ----- def
a_star_misplaced(start, goal):
    open_list = []
    heapq.heappush(open_list, (misplaced_tiles(start,
goal), 0, start, [start]))
    closed = set()

    while open_list:
        f, g, state, path = heapq.heappop(open_list)

        if state == goal:
            return path # solution found

        if state in closed:
            continue
        closed.add(state)

        for neighbor in get_neighbors(state):
            if neighbor not in closed:
                g_new = g + 1
                h_new =
misplaced_tiles(neighbor, goal)
                f_new =
g_new + h_new
                heapq.heappush(open_list, (f_new, g_new, neighbor, path + [neighbor]))
    return
None # no solution found

# ----- Run Example ----- if
__name__ == "__main__":
    start = (1, 2, 3,
4, 5, 6,

```



```

0, 7, 8)

goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

solution = a_star_misplaced(start, goal)

if solution:
    print("Solution found in", len(solution)-1, "moves.")
for step in solution:
    for i in range(0, 9, 3):
        print(step[i:i+3])
    print("-----")
else:
    print("No solution found.")

```

Solution found in 2 moves.

```

(1, 2, 3)
(4, 5, 6)
(0, 7, 8)

```

```

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

```

```

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

```

Manhattan distance – 8 puzzle import
heapq

```

# ----- Heuristic: Manhattan Distance ----- def
manhattan_distance(state, goal):
    """Sum of Manhattan distances of each tile from its goal position."""
    distance = 0
    for i, tile in enumerate(state):
        if tile != 0: # skip the
            blank      goal_pos = goal.index(tile)      distance += abs(i // 3 -
            goal_pos // 3) + abs(i % 3 - goal_pos % 3)    return distance

# ----- Puzzle Neighbors ----- def
get_neighbors(state):

```

```

    neighbors = []    idx =
state.index(0) # blank position    x, y
= divmod(idx, 3)

    moves = [(-1,0),(1,0),(0,-1),(0,1)] # up, down, left, right
for dx, dy in moves:
    nx, ny = x + dx, y + dy    if
0 <= nx < 3 and 0 <= ny < 3:
        new_idx = nx * 3 + ny        new_state = list(state)
new_state[idx], new_state[new_idx] = new_state[new_idx], new_state[idx]
neighbors.append(tuple(new_state))    return neighbors

# ----- A* Search ----- def
a_star_manhattan(start, goal):
    open_list = []    heapq.heappush(open_list, (manhattan_distance(start,
goal), 0, start, [start]))    closed = set()

    while open_list:
        f, g, state, path = heapq.heappop(open_list)

        if state == goal:
            return path # solution found

        if state in closed:
            continue
        closed.add(state)

        for neighbor in get_neighbors(state):
            if neighbor not in closed:
                g_new = g + 1        h_new =
manhattan_distance(neighbor, goal)        f_new =
g_new + h_new
                heapq.heappush(open_list, (f_new, g_new, neighbor, path + [neighbor]))    return
None # no solution found

# ----- Run Example ----- if
__name__ == "__main__":
    start = (1, 2, 3,
4, 5, 6,
        0, 7, 8)

    goal = (1, 2, 3,
4, 5, 6,
        7, 8, 0)

    solution = a_star_manhattan(start, goal)

```

```

if solution:
    print("Solution found in", len(solution)-1, "moves.")
for step in solution:
    for i in range(0, 9, 3):
        print(step[i:i+3])
        print("-----")
    else:
        print("No solution found.")

```

Output:

```

Solution found in 2 moves.
(1, 2, 3)
(4, 5, 6)
(0, 7, 8)
-----
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)
-----
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
-----

```

```

Iterative Deepening Depth First Search #
----- Depth Limited Search -----
def DLS(graph, node, goal, limit, visited):
    if node == goal:
        return True
    if limit == 0:
        return False

    visited.add(node)
    for neighbor in graph.get(node, []):
        if neighbor not in visited:
            if DLS(graph, neighbor, goal, limit - 1, visited):
                return True
    return False

# ----- IDDFS -----
def IDDFS(graph, start, goal, max_depth):
    for depth in range(max_depth + 1):
        visited = set()
        if DLS(graph, start, goal, depth, visited):
            return True
    return False

```

```
# ----- Example Run ----- if
__name__ == "__main__":
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

start = 'A'
goal = 'F'

if IDDFS(graph, start, goal, max_depth=3):
print(f"Goal {goal} found within depth limit.") else:
    print(f"Goal {goal} not found within depth limit.")
```

Output:

```
if __name__ == "__main__":
    graph = {
        'A': ['B', 'C'],
        'B': ['D', 'E'],
        'C': ['F'],
        'D': [],
        'E': ['F'],
        'F': []
    }

    start = 'A'
    goal = 'F'

    if IDDFS(graph, start, goal, max_depth=3):
        print(f"Goal {goal} found within depth limit.")
    else:
        print(f"Goal {goal} not found within depth limit.")
```

Goal F found within depth limit.

Program 3

Implement A* search algorithm

Algorithm

Lab-3

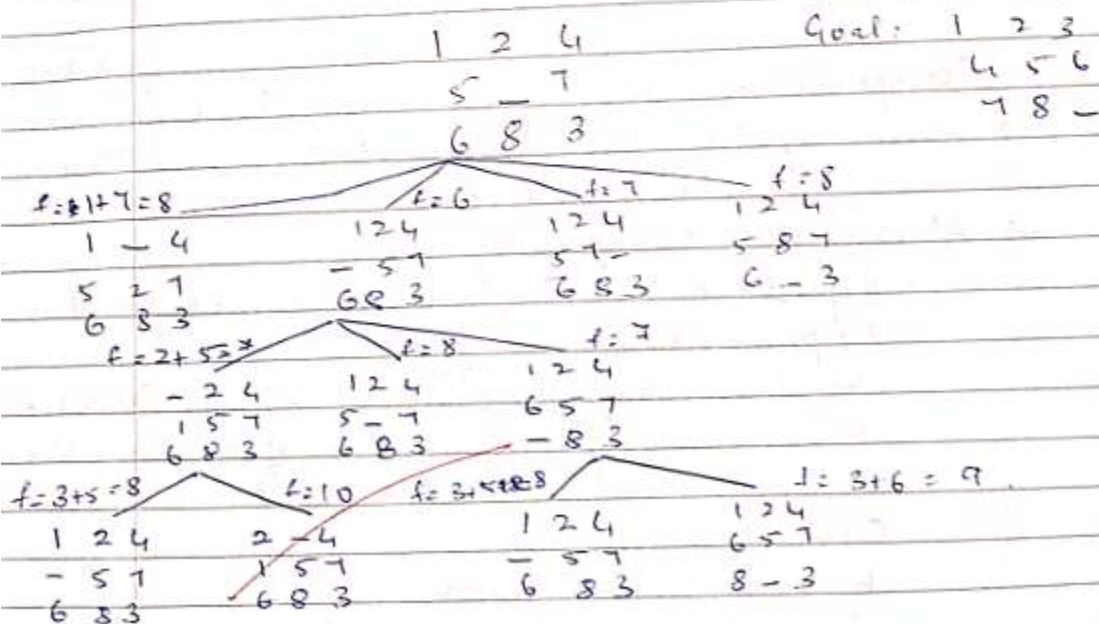
8 Puzzle using A*

Algorithm Solve8Puzzle - Astar (start state, goal state)

Input: start state, goal state

Output: sequence of moves from start state to goal state

- * Take the start state, ^{cycle} for each possible moves, the number of misplaced tiles
- * $f(n) = g(n) + h(n) \sim f = g + h$
- $g(n) \leftarrow$ the level (no. of moves made)
- $h(n) \leftarrow$ no. of misplaced tiles.
- * for the least $f \rightarrow$ repeat the steps until it reaches the goal state



[Tried $\rightarrow$ 21 steps]

Handwritten notes on a notebook page comparing A* and IDDFS approaches for the 8-puzzle problem.

Diagram: A search tree starting from the root state [1, 2, 3, 4, 5, 6, 7, 8, 0]. The tree shows various states and their children, with some states marked as goal states (e.g., [1, 2, 3, 4, 5, 6, 7, 8, 0]).

Comparison:

- Despite having worst case as $O(b^d)$ in both, A* performs better and provides most promising paths.
- Meanwhile IDDFS doesn't use heuristic and leads to exploration of many states (visiting a lot of states).
- A* best → Manhattan distance.
- IDDFS worst → IDDFS.

Computing the time complexities

→ State space tree alone can have 9! possible states (permutations).

→ A* search complexity

- depends on no. of nodes generated and expanded
- ↳ branching factor: 4 moves (U, D, L, R)
- ↳ heuristic function: no. of misplaced tiles

$O(b^d)$: worst case

b : branching factor

d : depth

Time taken:

- A*: 0.002896 seconds
- IDDFS: 0.321174 seconds
- Manhattan: 0.000664 seconds
- Misplaced tiles: 0.004026 seconds

Code:

```
import heapq
```

Goal state for the 8 puzzle

```
goal_state = [[1, 2, 3],
```

```
              [4, 5, 6],
```

```
              [7, 8, 0]] # 0 is the blank space
```

Directions: up, down, left, right

```
moves = [(1, 0), (-1, 0), (0, 1), (0, -1)]
```

```
def manhattan_distance(state):
```

```
    """Calculate Manhattan distance heuristic for a given state."""
```

```
    distance = 0    for i in range(3):    for j in range(3):
```

```
    value = state[i][j]    if value != 0:
```

```

        target_x = (value - 1) // 3        target_y =
(value - 1) % 3        distance += abs(i - target_x) +
abs(j - target_y)    return distance

```

```

def find_blank(state):
    """Find the position of blank (0) in the puzzle."""
    for i in range(3):        for j in range(3):        if
state[i][j] == 0:
        return i, j

```

```

def state_to_tuple(state):
    """Convert list state to tuple (for hashing in sets)."""
    return tuple(tuple(row) for row in state)

```

```

def a_star(start_state):
    """A* algorithm to solve 8-puzzle."""    start_h =
manhattan_distance(start_state)    pq = [(start_h, 0,
start_state, [])] # (f, g, state, path)    visited = set()

```

```

    while pq:
        f, g, state, path = heapq.heappop(pq)

```

```

        if state == goal_state:
            return path + [state]

```

```

        visited.add(state_to_tuple(state))
        x, y = find_blank(state)

```

```

        for dx, dy in moves:
            new_x, new_y = x + dx, y + dy
            if 0 <= new_x < 3 and 0 <= new_y < 3:
                new_state = [row[:] for row in state] # deep copy
                # Swap blank with neighbor
                new_state[x][y], new_state[new_x][new_y] =
new_state[new_x][new_y], new_state[x][y]

```

```

                if state_to_tuple(new_state) not in visited:
                    h
                    = manhattan_distance(new_state)                    heapq.heappush(pq,
(g + 1 + h, g + 1, new_state, path +
[state]))

```

```

    return None # No solution found

```



```

# Example usage:
start_state = [[1, 2, 3],
               [4, 5, 6],
               [0, 7, 8]]

solution = a_star(start_state)

print("Steps to reach the goal:")
for step in solution:
    for row
in step:
        print(row)
print()

```

Output:

```

Steps to reach the goal:
[1, 2, 3]
[4, 5, 6]
[0, 7, 8]

[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]

```

Program 4

Implement Hill Climbing search algorithm to solve N-Queens problem

Algorithm:

Lab 4
4 Queens Hill Climbing.

Start:
current ← initial state
while current.value >= 0
 neighbours ← value of the possible
 next state
 while current.value >= neighbours.value
 do
 neighbour ← highest next possible
 value
 if neighbour.value < current.value
 then return current.value
 current ← neighbour

4 Queens problem
Output: 0 3 0 0
 0 0 0 1
 1 0 0 0
 0 0 1 0

For given example, $x_0=3, x_1=1, x_2=2, x_3=0$
Initial state = [3, 1, 2, 0], Cost = 2

Neighbours and costs:

State = [1, 1, 2, 0]	Cost = 2
State = [3, 0, 2, 0]	Cost = 2
State = [3, 1, 2, 2]	Cost = 2
State = [3, 1, 3, 0]	Cost = 2
State = [2, 1, 1, 0]	Cost = 3
State = [2, 1, 0, 0]	Cost = 3
State = [3, 1, 2, 1]	Cost = 3
State = [2, 2, 2, 0]	Cost = 3
State = [0, 1, 2, 0]	Cost = 4
State = [3, 1, 1, 0]	Cost = 4
State = [3, 1, 2, 3]	Cost = 4
State = [3, 2, 2, 0]	Cost = 4

State in local minimum.
Final state = [3, 1, 2, 0] Final cost = 2

Code:

```
import random
```

Heuristic: number of pairs of queens attacking each other

```
def calculate_cost(state): cost = 0
```

```
    n = len(state) for i in range(n): for j in range(i + 1,
```

```
    n): if state[i] == state[j] or abs(state[i] - state[j]) == abs(i
```

```
    - j): cost += 1
```

```
    return cost
```

```

# Generate all neighbors of the current state
def get_neighbors(state):    neighbors = []
    n = len(state)
    for col in range(n):
    for row in range(n):
        if state[col] != row: # move queen in col to new row
            neighbor = state.copy()
            neighbor[col] = row
            neighbors.append(neighbor)
    return neighbors

# Hill climbing algorithm def
hill_climb(initial_state):
    current = initial_state
    current_cost = calculate_cost(current)

    print(f"Initial state: {current}, Cost = {current_cost}")

    while True:
        neighbors = get_neighbors(current)
        best_neighbor = None
        best_cost = current_cost

        # Find the best neighbor
        for neighbor in neighbors:
            cost = calculate_cost(neighbor)
            if cost < best_cost:
                best_cost = cost
                best_neighbor = neighbor

        # If no better neighbor is found → stop        if
        best_neighbor is None:            print(f"Final state: {current},
        Cost = {current_cost}")            return current, current_cost

        # Move to the better neighbor        current,
        current_cost = best_neighbor, best_cost
        print(f"Move to: {current}, Cost = {current_cost}")

# Example usage if
__name__ == "__main__":
    n = int(input("Enter number of queens (N): "))
    print("Enter initial state as space-separated row positions for each column.")
    print("Example for N=4: '1 3 0 2' means queen at (0,1), (1,3), (2,0), (3,2).")

```

```

initial_state = list(map(int, input("Initial state: ").split()))

if len(initial_state) != n:
    print("Invalid input: Length of initial state must be N.")
else:
    solution, cost = hill_climb(initial_state)

    if cost == 0:
        print("Goal state reached!")
    else:
        print("Stuck in local minimum.")

```

Output:

```

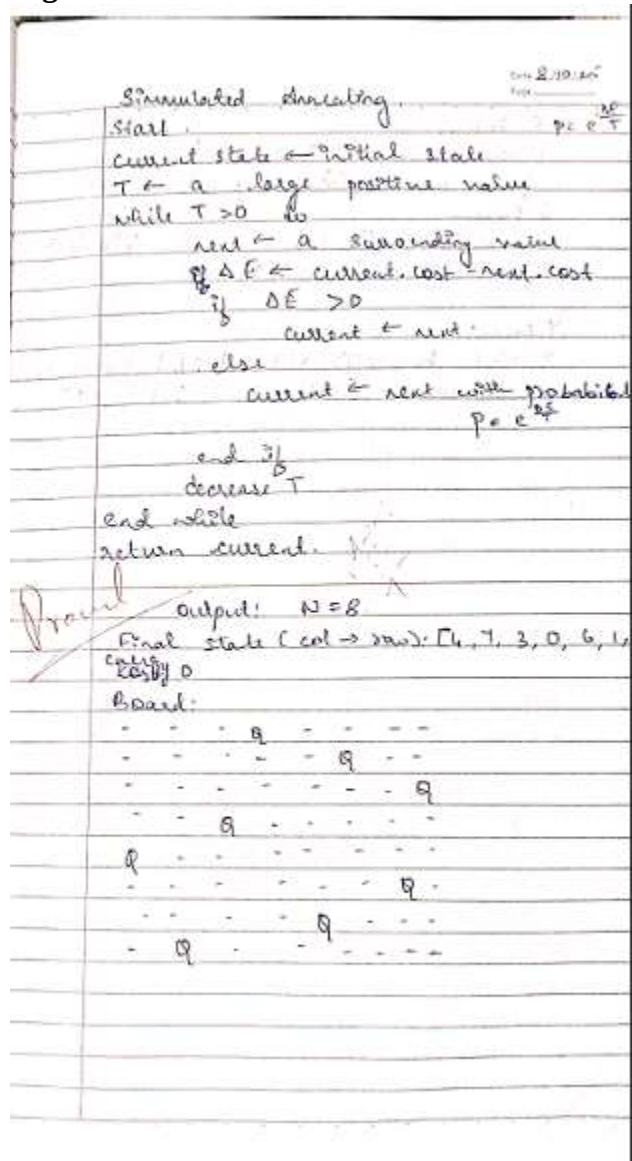
Enter number of queens (N): 4
Enter initial state as space-separated row positions for each column.
Example for N=4: '1 3 0 2' means queen at (0,1), (1,3), (2,0), (3,2).
Initial state: 3 1 2 0
Initial state: [3, 1, 2, 0], Cost = 2
Final state: [3, 1, 2, 0], Cost = 2

```

Program 5

Simulated Annealing to Solve 8-Queens problem

Algorithm:



Code:

```
import random
import math
```

```
def cost(state, N):
    """Compute number of attacking queen pairs.""" conflicts = 0
    for i in range(N):
        for j in range(i+1, N):
            if state[i] == state[j] or abs(state[i] - state[j]) == abs(i - j):
                conflicts += 1
    return conflicts
```

```

def random_neighbor(state, N):
    """Generate a neighbor by moving one queen to another row in a random column."""
    neighbor = state.copy()    col = random.randrange(N)    new_row =
    random.randrange(N-1)    if new_row >= neighbor[col]:
        new_row += 1
    neighbor[col] = new_row
    return neighbor

def simulated_annealing(N, T0=5.0, alpha=0.995, Tmin=1e-6, max_iters=50000):
    """Solve N-Queens using simulated annealing."""
    # Random initial state: one queen per column    state
    = [random.randrange(N) for _ in range(N)]
    current_cost = cost(state, N)
    T = T0
    it = 0

    while T > Tmin and it < max_iters and current_cost != 0:
        neighbor = random_neighbor(state, N)
        neighbor_cost = cost(neighbor, N)
        delta = neighbor_cost - current_cost

        if delta <= 0 or random.random() < math.exp(-delta / T):
            state, current_cost = neighbor, neighbor_cost

        T *= alpha
        it += 1

    return state, current_cost

def print_board(state, N):
    """Pretty-print the board with row and column numbers."""
    print(" " + " ".join(str(c) for c in range(N))) # column indices
    for r in range(N):
        row = f"{{r}} " # row index
        for c in range(N):
            row += "Q " if state[c] == r else ". "
        print(row)
        print()

# -----
# User input
# -----
N = int(input("Enter number of queens (N): "))

solution, c = simulated_annealing(N)

```

```
print(f"\nFinal state (col -> row): {solution}")
print("Cost:", c) print("\nBoard:")
print_board(solution, N)
```

Output:

```
Enter number of queens (N): 5

Final state (col -> row): [4, 1, 3, 0, 2]
Cost: 0

Board:
  0 1 2 3 4
0 . . . Q .
1 . Q . . .
2 . . . . Q
3 . . Q . .
4 Q . . . .
```

Program 6

Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.

Algorithm:

Date 15/10/25
Page

Propositional entailment

Algorithm:

function TT-ENTAILS? (KB, α) returns true or false
inputs: KB, the knowledge base, a sentence in propositional logic α , the query, a sentence in propositional logic
symbols $\leftarrow$ a list of the proposition symbols in KB and α
return TT-CHECK-ALL (KB, α , symbols, $\{ \}$)

function TT-CHECK-ALL (KB, α , symbols, model)
returns true or false
if EMPTY? (symbols) then
if PL-TRUE? (KB, model) then return
return PL-TRUE? (α , model)
else return true // when KB is false, always return true
else do
P $\leftarrow$ FIRST (symbols)
rest $\leftarrow$ REST (symbols)
return TT-CHECK-ALL (KB, α , rest, model) $\cup$ {P = true}
and
return TT-CHECK-ALL (KB, α , rest, model) $\cup$ {P = false}


```

models = [] for P, Q,
R in values:
    s1 = implies(Q, P)
s2 = implies(P, not Q)
s3 = Q or R
    kb_val = s1 and s2 and s3
    print(f"{P}\t{Q}\t{R}\t{s1}\t{s2}\t{s3}\t{kb_val}")
if kb_val:
    models.append((P, Q, R))

print("\n Models where KB is True:", models)

# Check entailments entails_R = all(R for P, Q, R in
models) entails_R_imp_P = all((not R) or P for P, Q, R
in models)
entails_Q_imp_R = all((not Q) or R for P, Q, R in models)

print("\nEntailments:") print("KB  $\models$  R :",
entails_R) print("KB  $\models$  R  $\rightarrow$  P :",
entails_R_imp_P) print("KB  $\models$  Q  $\rightarrow$  R
:", entails_Q_imp_R)

```

Output:

P	Q	R	$Q \rightarrow P$	$P \rightarrow \neg Q$	$Q \vee R$	KB
False	False	False	True	True	False	False
False	False	True	True	True	True	True
False	True	False	False	True	True	False
False	True	True	False	True	True	False
True	False	False	True	True	False	False
True	False	True	True	True	True	True
True	True	False	True	False	True	False
True	True	True	True	False	True	False

```
Models where KB is True: [(False, False, True), (True, False, True)]
```

```
Entailments:
```

```
KB  $\models$  R : True
```

```
KB  $\models$  R  $\rightarrow$  P : False
```

```
KB  $\models$  Q  $\rightarrow$  R : True
```

Program 7

Implement unification in first order logic.

Algorithm:

Find $\theta(m, n, \psi)$

Step 1: If ψ is a variable or constant, then:

- If ψ is a variable, then return NIL
- If ψ is a constant, then return NIL

Step 2: If ψ is a variable, then:

- If ψ occurs in ψ , then return FAILURE
- Else return $\{\psi / \psi\}$

Step 3: If the initial predicate symbols in ψ and ψ are not same, then return FAILURE

Step 4: If ψ and ψ have a different no. of arguments, then return FAILURE

Step 5: Set substitution set (SUBST) to NIL

Step 6: For $i = 1$ to the number of elements in ψ :

- Call unify function with the i th element ψ_i and i th element of ψ and put the result into S .
- If $S = \text{failure}$ then return FAILURE
- If $S \neq \text{NIL}$ then do:
 - Apply S to the remainder of both ψ and ψ
 - SUBST = APPEND (S , SUBST)

Step 7: Return SUBST

Find $\theta(m, n, \psi)$

① $P(f(x), g(y), y)$
 $P(f(g(z)), g(f(a)), f(a))$

$\theta = \{x = f(a), y = g(z), y = f(a)\}$ unification success

② $\phi(x, f(x))$
 $\phi(f(y), y)$

$x \rightarrow f(y)$
 $f(x) \rightarrow y$ Not possible unification fails

③ $H(x, g(x))$
 $H(g(y), g(g(z)))$

$x \rightarrow g(y)$
 $g(x) \rightarrow g(g(z))$

$g(x) \rightarrow g(g(y))$
 $g(g(y)) \rightarrow g(g(z))$

$g(y) \rightarrow g(z) \rightarrow y = z$ unification success

Result $\theta = \{x = g(y), y = z\}$ unification success

Code:

```
def unify(x, y, substitutions=None):
    if substitutions is None:
        substitutions = {}
```

```
    # If both are identical
    if x == y:
        return substitutions
```

```
    # If x is a variable    if isinstance(x,
    str) and x.islower():    return
    unify_var(x, y, substitutions)
```

```

    # If y is a variable    if isinstance(y,
str) and y.islower():      return
unify_var(y, x, substitutions)

    # If both are compound expressions (like lists or tuples)
if isinstance(x, tuple) and isinstance(y, tuple):    if x[0]
!= y[0] or len(x) != len(y):
    return None    for a,
b in zip(x[1:], y[1:]):
    substitutions = unify(a, b, substitutions)
if substitutions is None:
    return None
    return substitutions

return None

def unify_var(var, x, substitutions):
if var in substitutions:
    return unify(substitutions[var], x, substitutions)
elif x in substitutions:
    return unify(var, substitutions[x], substitutions)
elif occurs_check(var, x, substitutions):
    return None
else:
    substitutions[var] = x
    return substitutions

def occurs_check(var, x, substitutions):
if var == x:    return True    elif
isinstance(x, tuple):
    return any(occurs_check(var, arg, substitutions) for arg in x[1:])    elif isinstance(x, str)
and x in substitutions:
    return occurs_check(var, substitutions[x], substitutions)
return False

# Example expr1 = ("Eats",
"x", "Apple")
expr2 = ("Eats", "Riya", "y")

result = unify(expr1, expr2)
print("Unification:", result)

```

Output:

```
Unification: {'x': 'Riya', 'y': 'Apple'}
```


Program 8

Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.

Algorithm:

For Forward Chaining Algorithm

function FOL-FC-ASK (KB, α) returns a Substitution

Define

Inputs: KB, the knowledge base, a set of first-order definite clauses

α , the query, an atomic sentence

local variables: new, the new sentences inferred on each iteration

repeat until new is empty

 new $\leftarrow \{\}$

 for each rule in KB do

$(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE VARIABLES}$

 for each θ such that $\text{SUBST}(\theta, p_1 \wedge \dots \wedge p_n) = \text{SUBST}(\theta, p_1 \wedge \dots \wedge p_n)$

 for some $p_1, \dots, p_n$ in KB

$q' \leftarrow \text{SUBST}(\theta, q)$

 if q' does not unify with some sentence already in KB or new then

 add q' to new

$\phi \leftarrow \text{UNIFY}(q', \alpha)$

 if ϕ is not fail then return ϕ

 add new to KB

return false.

Considering the following problem:

As per the law, it is a crime for an American to sell weapons to hostile nations. Country A, an enemy of America, has some missiles, and all the missiles will sold to it by Robert, who is an American citizen.

Prove that "Robert is Criminal".

Output:

Inferred: Hostile(CountryA)

Inferred: Criminal(Robert)

Final Facts:

American(Robert)

Enemy(CountryA, America)

OwnsMissiles(CountryA)

Sells(Robert, Missiles, CountryA)

Conclusion: Robert is a Criminal

Code:

```
def forward_chaining(KB, query):
    inferred = set()
    new_inferred = True

    while new_inferred:
        new_inferred = False
        for rule in KB:
            premises, conclusion = rule
            if all(p in inferred or p in KB for p in premises) and conclusion not in inferred:
                inferred.add(conclusion)
        new_inferred = True
    if conclusion == query:
```

```

        return True
    return False

# Example Knowledge Base
KB = [
    ("American(Robert)", "Weapon(x)", "Sells(Robert, x, A)", "Hostile(A)"),
    "Criminal(Robert)",
    ("Missile(x)", "Weapon(x)"),
    ("Owns(A, x)", "Missile(x)", "Sells(Robert, x, A)"),
    ("Enemy(A, America)", "Hostile(A)")
]

facts = {
    "American(Robert)",
    "Enemy(A, America)",
    "Owns(A, T1)",
    "Missile(T1)"
}
# Add base facts to KB
for fact in facts:
    KB.append([fact])
# Query
query = "Criminal(Robert)"
print("Is Robert a criminal?", forward_chaining(KB, query))

```

Output:

```
Is Robert a criminal? False
```

Program 9

Create a knowledge base consisting of first order logic statements and prove the given query using Resolution.

Algorithm:

Date / /
Page

Resolution from FOL

Algorithm to convert Logic Statement to CNF

1. Eliminate Biconditionals and Implications:
 - Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$
 - Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg \alpha \vee \beta$.
2. Move $\neg$ inwards:
 - $\neg(\forall x p) \equiv \exists x \neg p$,
 - $\neg(\exists x p) \equiv \forall x \neg p$,
 - $\neg(\alpha \vee \beta) \equiv \neg \alpha \wedge \neg \beta$,
 - $\neg(\alpha \wedge \beta) \equiv \neg \alpha \vee \neg \beta$,
 - $\neg \neg \alpha \equiv \alpha$.
3. Standardize variables apart by renaming them
each quantifier should use a different variable
4. Skolemize: each existential variable is replaced by a Skolem constant or Skolem function of the enclosing universally quantified variables.
 - For instance, $\exists x \text{Rich}(x)$ becomes $\text{Rich}(c_1)$ where c_1 is a new Skolem constant.
 - "Everyone has a heart" $\forall x \text{Person}(x) \Rightarrow \exists y \text{Heart}(y) \wedge \text{Has}(x, y)$
becomes $\forall x \text{Person}(x) \Rightarrow \text{Heart}(H(x)) \wedge \text{Has}(x, H(x))$
where H is a new symbol (Skolem function).
5. Drop universal quantifiers
 - For instance, $\forall x \text{Person}(x)$ becomes $\text{Person}(x)$
6. Distribute $\wedge$ over $\vee$:
 - $(\alpha \wedge \beta) \vee \gamma \equiv (\alpha \vee \gamma) \wedge (\beta \vee \gamma)$

Output:

Original Expression:
Implies $(P, q | \neg R)$

CNF form:
 $q | \neg P | \neg R$

Code:

```

import copy
# -----
# Predicate Structure
# -----
class Predicate:
def __init__(self, name, args, negated=False):
    self.name = name    self.args = args
    if isinstance(args, tuple) else tuple(args)    self.negated =
negated

    def __eq__(self, other):
        return (self.name == other.name and
self.args == other.args and
                self.negated == other.negated)

    def __hash__(self):
        return hash((self.name, self.args, self.negated))

    def __repr__(self):
        neg = "~" if self.negated else ""
args_str = ",".join(str(a) for a in self.args)
        return f"{neg}{self.name}({args_str})"

    def negate(self):
        return Predicate(self.name, self.args, not self.negated)

    def substitute(self, theta):
        """Apply substitution theta to this predicate"""
        new_args = tuple(substitute_term(arg, theta) for arg in self.args)
        return Predicate(self.name, new_args, self.negated)

def substitute_term(term, theta):
    """Apply substitution to a term"""
    if isinstance(term, str) and term.islower(): # variable
        if term in theta:
            return substitute_term(theta[term], theta)
        return term
    elif isinstance(term, tuple):
        return tuple(substitute_term(t, theta) for t in term)
    return term

# -----
# Unification Algorithm
# -----
def unify(x, y, theta=None):
    if theta is None:
        theta = {}
    if theta == "FAIL":

```

```

        return "FAIL"    elif x == y:        return theta
    elif isinstance(x, str) and x.islower(): # variable
    return unify_var(x, y, theta)    elif isinstance(y,
    str) and y.islower(): # variable        return
    unify_var(y, x, theta)    elif isinstance(x, tuple)
    and isinstance(y, tuple):        if len(x) != len(y):
    return "FAIL"        theta = unify(x[0], y[0], theta)
    if theta == "FAIL":            return "FAIL"
    return unify(x[1:], y[1:], theta)    else:
        return "FAIL"

def unify_var(var, x, theta):
    if var in theta:
        return unify(theta[var], x, theta)    elif
    isinstance(x, str) and x.islower() and x in theta:
        return unify(var, theta[x], theta)
    elif occurs_check(var, x, theta):
        return "FAIL"
    else:
        new_theta = copy.deepcopy(theta)
    new_theta[var] = x
    return new_theta

def occurs_check(var, x, theta):    if var == x:
    return True    elif isinstance(x, str) and x.islower()
    and x in theta:
        return occurs_check(var, theta[x], theta)
    elif isinstance(x, tuple):
        return any(occurs_check(var, xi, theta) for xi in x)
    return False

# -----
# Variable Standardization
# -----
var_counter = 0

def standardize_variables(clause):
    """Rename all variables in a clause to unique names"""
    global var_counter    mapping = {}    new_clause = []

    for pred in clause:
    new_args = []        for
    arg in pred.args:
        if isinstance(arg, str) and arg.islower(): # variable
        if arg not in mapping:

```

```

        mapping[arg] =
f"{arg}{var_counter}"          var_counter +=
1          new_args.append(mapping[arg])
else:
        new_args.append(arg)
new_clause.append(Predicate(pred.name, new_args, pred.negated))

    return new_clause

# -----
# Resolution Algorithm
# ----- def
resolve(ci, cj):
    """Resolve two clauses using FOL resolution"""
    ci = standardize_variables(ci)
    cj = standardize_variables(cj)

    resolvents = []

    for i, pi in enumerate(ci):
    for j, pj in enumerate(cj):
        # Check if predicates can be resolved (opposite signs, same name)
        if pi.negated != pj.negated and pi.name == pj.name:
            # Try to unify the arguments
            theta = unify(pi.args, pj.args)
            if
            theta != "FAIL":
                # Create resolvent by removing resolved predicates and applying substitution
                new_clause = []

                # Add literals from ci except
                pi
                for k, pred in enumerate(ci):
                if k != i:
                    new_clause.append(pred.substitute(theta))

                # Add literals from cj except
                pj
                for k, pred in enumerate(cj):
                if k != j:
                    new_clause.append(pred.substitute(theta))

                # Remove duplicates
                new_clause = list(set(new_clause))
                resolvents.append(new_clause)

    return resolvents

```

```

def fol_resolution(kb, query):
    """FOL resolution algorithm"""
    # Negate query and add to KB
    clauses = [clause[:] for clause in kb] # deep copy
    clauses.append([query.negate()])

    print(f"\nKnowledge Base + Negated Query:")
    for i, clause in enumerate(clauses):
        print(f" {i+1}. {clause}")
    print()

    iteration = 0
    while True:
        iteration += 1        n = len(clauses)        pairs = [(clauses[i],
        clauses[j]) for i in range(n) for j in range(i + 1, n)]

        new_clauses = []
        for (ci, cj) in pairs:
            resolvents = resolve(ci, cj)

            for resolvent in resolvents:
                if len(resolvent) == 0:
                    print(f"Iteration {iteration}: Derived empty clause from:")
                    print(f" {ci}")                print(f" {cj}")                print(" → []")
                    (Contradiction found!)")                return True

            # Check if this is a new clause                if resolvent
            not in clauses and resolvent not in new_clauses:
                new_clauses.append(resolvent)

        if not new_clauses:
            print(f"Iteration {iteration}: No new clauses derived. Query cannot be proved.")
            return False

        print(f"Iteration {iteration}: Generated {len(new_clauses)} new clause(s)")
        for clause in new_clauses:
            clauses.append(clause)

    # -----
    # Example Usage
    # -----
    if __name__ == "__main__":
        # Define knowledge base
        kb = [
            # John likes all food: Food(x) => Likes(John, x)
            [Predicate("Food", ("x",), negated=True), Predicate("Likes", ("John", "x"))],

```

```

# Food(Apple)
[Predicate("Food", ("Apple",))],

# Food(Vegetables)
[Predicate("Food", ("Vegetables",))],

# Eats(Anil, Peanuts)
[Predicate("Eats", ("Anil", "Peanuts"))],

# Alive(Anil)
[Predicate("Alive", ("Anil",))],

# If alive and eats something, that thing is food:  $\text{Alive}(x) \wedge \text{Eats}(x,y) \Rightarrow \text{Food}(y)$ 
[Predicate("Alive", ("x",), negated=True),
 Predicate("Eats", ("x", "y"), negated=True),
 Predicate("Food", ("y",))],

# Harry eats everything Anil eats:  $\text{Eats}(\text{Anil}, y) \Rightarrow \text{Eats}(\text{Harry}, y)$ 
[Predicate("Eats", ("Anil", "y"), negated=True),
 Predicate("Eats", ("Harry", "y"))]
]

# Query: Does John like Peanuts?
query = Predicate("Likes", ("John", "Peanuts"))

print("=" * 60)
print("FIRST-ORDER LOGIC RESOLUTION THEOREM
PROVER") print("=" * 60) print(f"\nQuery: {query}")
print("-" * 60)

result = fol_resolution(kb, query)

print("\n" + "=" * 60)
if result:
    print("Query is PROVED using resolution!") else:
    print("Query CANNOT be proved.") print("="
* 60)

```

Output:

```
=====
FIRST-ORDER LOGIC RESOLUTION THEOREM PROVER
=====
```

```
Query: Likes(John,Peanuts)
-----
```

```
Knowledge Base + Negated Query:
```

1. [$\sim$ Food(x), Likes(John,x)]
2. [Food(Apple)]
3. [Food(Vegetables)]
4. [Eats(Anil,Peanuts)]
5. [Alive(Anil)]
6. [$\sim$ Alive(x), $\sim$ Eats(x,y), Food(y)]
7. [$\sim$ Eats(Anil,y), Eats(Harry,y)]
8. [$\sim$ Likes(John,Peanuts)]

```
Iteration 1: Generated 8 new clause(s)
```

```
Iteration 2: Generated 16 new clause(s)
```

```
Iteration 3: Derived empty clause from:
```

```
  [Eats(Anil,Peanuts)]
```

```
  [ $\sim$ Eats(Anil,Peanuts)]
```

```
  → [] (Contradiction found!)
```

```
=====
✅ Query is PROVED using resolution!
=====
```

Program 10

Implement Alpha-Beta Pruning.

Algorithm:

Alpha - Beta search algorithm:

function ALPHA-BETA-SEARCH(state) returns an action

$v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$

return the action in $\text{ACTIONS}(\text{state})$ with value v

function MAX-VALUE(state, α , β) returns a utility value

if $\text{TERMINAL-TEST}(\text{state})$ then return $\text{UTILITY}(\text{state})$

$v \leftarrow -\infty$

for each a in $\text{ACTIONS}(\text{state})$ do

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULTS}(s, a), \alpha, \beta))$

if $v \geq \beta$ then return v

$\alpha \leftarrow \text{MAX}(\alpha, v)$

return v

function MIN-VALUE(state, α , β) returns a utility value

if $\text{TERMINAL-TEST}(\text{state})$ then return $\text{UTILITY}(\text{state})$

$v \leftarrow +\infty$

for each a in $\text{ACTIONS}(\text{state})$ do

$v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$

if $v \leq \alpha$ then return v

$\beta \leftarrow \text{MIN}(\beta, v)$

return v

$\alpha \beta$ $\uparrow$ $\uparrow$ $\uparrow$
min

Code:

```

import math
# Alpha-Beta Pruning Algorithm
def alpha_beta(depth, node_index, maximizing_player,
               values, alpha, beta, max_depth, path, pruned):
    # Base case: leaf node
    if depth == max_depth:
        return values[node_index], [node_index]

    if maximizing_player:
        best = -math.inf
    best_path = []
    for i in range(2): # two children per node
        val, child_path = alpha_beta(depth + 1, node_index * 2 + i, False, values, alpha, beta, max_depth, path, pruned)
        if val > best:
            best = val
            best_path = [node_index] + child_path
            alpha = max(alpha, best)
        if beta <= alpha:
            pruned.append((node_index, "Right" if i == 0 else "Left"))
        break
    return best, best_path
else:
    best = math.inf
    best_path = []
    for i in range(2):
        val, child_path = alpha_beta(depth + 1, node_index * 2 + i, True, values, alpha, beta, max_depth, path, pruned)
        if val < best:
            best = val
            best_path = [node_index] + child_path
            beta = min(beta, best)
        if beta <= alpha:
            pruned.append((node_index, "Right" if i == 0 else "Left"))
        break
    return best, best_path

# Example usage if
__name__ == "__main__":
    # Example game tree (leaf node values)
    values = [3, 5, 6, 9, 1, 2, 0, -1]

    print("Leaf Node Values:", values)
    path = []
    pruned = []

    max_depth = 3
    result, best_path = alpha_beta(0, 0, True, values, -math.inf, math.inf, max_depth, path, pruned)

```

```
print("\nOptimal Value at Root Node:",  
result) print("Best Path (Node Indices):",  
best_path) print("Pruned Nodes:", pruned)
```

Output:

```
Leaf Node Values: [3, 5, 6, 9, 1, 2, 0, -1]  
  
Optimal Value at Root Node: 5  
Best Path (Node Indices): [0, 0, 0, 1]  
Pruned Nodes: [(1, 'Right'), (1, 'Right')]
```